Comparison of AdamW, Muon and Hybrid Optimization for Full Fine-Tuning of Qwen2.5-0.5B

Ilya Saenko
sia3301@yandex.ru

2026

Abstract

Optimizers based on orthogonal gradient updates, such as Muon, have recently emerged as promising alternatives to AdamW for training large-scale language models. However, their behavior under resource-constrained conditions small model sizes, limited batch sizes, and short training budgets remains underexplored. In this work, we conduct a systematic empirical comparison of three optimization strategies for continued pre-training of the Qwen2.5-0.5B language model: (1) a standard AdamW baseline, (2) a Moonlight Muon configuration that applies orthogonal gradient updates to internal 2D weight matrices while using AdamW for remaining parameters, and (3) a depth-stratified Hybrid approach that restricts Muon to the first 12 transformer layers and delegates deeper layers and scalar parameters to AdamW. All methods are evaluated over 1500 training steps on the OpenWebText corpus and assessed zero-shot on five reasoning benchmarks: ARC-Challenge, ARC-Easy, HellaSwag, PIQA, and Winogrande. Our results show that all three optimizers converge at nearly identical rates and achieve comparable downstream accuracy, with the Hybrid approach reaching the highest aggregate score of 0.5260. Despite the overall parity in performance, neither Muon nor the Hybrid method introduces training instability, confirming their safe applicability as drop-in replacements for AdamW even under small-batch, small-model regimes. We discuss the contributing factors to this parity namely gradient noise at small batch sizes and the well-conditioned optimization landscape of a compact model.

1 Introduction

Optimizer choice critically impacts language model training outcomes. While AdamW has become the standard for transformer models, Muon-which applies orthogonalization to gradient matrices-has shown promise at scale. However, Muon's effectiveness in resource-constrained environments remains understudied.

This report compares three optimization strategies for continued pre-training of Qwen2.5-0.5B: standard AdamW, Moonlight Muon (applying Muon to 2D matrices and AdamW to other parameters), and a novel Hybrid approach (using Muon only in the first 12 transformer layers). Our controlled experiments maintain identical training conditions across all configurations.

Results show all three optimizers achieve comparable convergence rates and benchmark performance (within 0.3 percentage points). The Hybrid approach marginally outperforms both alternatives (0.5260 vs. 0.5254 for AdamW and 0.5230 for Muon). Our findings suggest that while Muon and Hybrid approaches are viable AdamW alternatives in resource-constrained settings, the orthogonalization benefits may be more pronounced with larger batch sizes where gradient noise is reduced.

2 Experiments

2.1 Experimental Setup

For all experiments, we use a single NVIDIA RTX 4090 GPU with 24 GB of VRAM and a system with 62 GB of RAM. To ensure reproducibility, we fix the random seed to 42 across all stochastic operations in Python, NumPy, and PyTorch. In addition, we enable deterministic algorithms in PyTorch and configure cuDNN for deterministic execution, at the cost of potentially reduced performance.

We choose Qwen2.5-0.5B as the base model for all experiments due to its moderate size, which allows for reasonable training times while remaining representative of modern language model architectures. The model is loaded in bfloat16 precision to improve memory efficiency and computational performance. We also enable gradient checkpointing to mitigate memory constraints and use scaled dot-product attention (SDPA) for transformer computations.

2.2 Training Configuration

All models are trained for 1500 steps with an effective batch size of 64 (a per-device batch size of 8 with 8 gradient accumulation steps). Each micro-batch loss is normalized by the number of gradient accumulation steps before the backward pass, so that the effective gradient magnitude remains consistent with a single-step batch of size 64. The maximum sequence length is fixed at 1024 tokens in all experiments. We use a cosine learning rate schedule with a warmup of 50 steps.

Training is performed with `bfloat16` automatic mixed precision (`torch.amp.autocast`) enabled. Since bfloat16 does not require loss scaling, the gradient scaler is disabled throughout all experiments.

No gradient clipping is applied (`max_grad_norm = None`), allowing us to observe the unmodified gradient dynamics of each optimizer and ensure a fair comparison.

For training data, we use the `Elriggs/openwebtext-100k` dataset, which provides a balanced and diverse corpus of web text.

2.3 Optimizers

To facilitate the use of multiple optimization strategies simultaneously, we implemented a straightforward `HybridOptimizer` wrapper class. It inherits from `torch.optim.Optimizer` and accepts a list of instantiated standard optimizers. Its primary function is to verify that the optimizers operate on strictly disjoint parameter sets, preventing parameter overlap, while synchronizing their `step` and `zero_grad` operations. Using this utility, we compare three optimization approaches:

1. **AdamW Baseline.** We use the standard PyTorch AdamW optimizer applied to all model parameters. The hyperparameters are set to a learning rate of 5×10^{-5}, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, and weight decay of 10^{-2}. This serves as our baseline, representing the de facto standard for training transformer-based language models.

2. **Moonlight Muon.** This approach utilizes the standard Muon optimizer from the `torch.optim` library in combination with AdamW via our `HybridOptimizer` wrapper. The model parameters are divided into two distinct groups. Muon is assigned exclusively to internal 2D weight matrices (excluding embeddings and language model heads). AdamW handles all remaining parameters, including 1D tensors, biases, LayerNorm weights, embeddings, and heads.

   For the Muon optimizer, we use a learning rate of 2×10^{-4}, momentum of 0.95 with Nesterov acceleration enabled, weight decay of 0.01, $\epsilon = 10^{-8}$, and 5 Newton-Schulz

iterations per step with coefficients $[3.4445, -4.775, 2.0315]$. The accompanying AdamW operates with the same settings as the baseline: learning rate of 5×10^{-5}, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, and weight decay of 10^{-2}.

3. **Hybrid Approach.** This configuration extends the Moonlight Muon strategy by partitioning the parameters into three groups, thereby utilizing three separate optimizers under the hood. The division is based on the network's depth:

   - **Muon** is applied to the 2D matrices of the first 12 transformer layers, utilizing the exact same hyperparameters as in the Moonlight Muon configuration.
   - **AdamW (2D)** is applied to the 2D matrices of the remaining layers (layer 12 and beyond).
   - **AdamW (Other)** is assigned to all remaining architectural components across the entire model (1D tensors, LayerNorms, biases, embeddings, and heads).

   Both AdamW instances in this setup share identical parameter settings: a learning rate of 5×10^{-5}, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, and a weight decay of 0.01. This depth-stratified design aims to leverage the fast convergence of Muon in the early layers while relying on the stability of AdamW for the deeper representations and sensitive 1D parameters.

Each optimizer is implemented with careful consideration of numerical stability and computational efficiency. All gradient operations use the same precision settings, and no gradient clipping is applied in order to ensure a fair comparison of optimizer dynamics.

2.4 Evaluation

To assess the impact of different optimization strategies on downstream model performance, we evaluate each checkpoint using a standardized benchmark suite. The evaluation is performed in a **zero-shot setting** to test the models' inherent reasoning and knowledge retrieval capabilities.

The benchmark suite includes the following tasks:

- **ARC-Challenge & ARC-Easy**: Grade-school science questions.
- **HellaSwag**: Commonsense reasoning for sentence continuation.
- **PIQA**: Physical interaction QA.
- **Winogrande**: Pronoun resolution and linguistic reasoning.

We report the standard accuracy for each task alongside the aggregate average score across all benchmarks.

Table 1: Final downstream evaluation results.

Method	ARC-C	ARC-E	HellaSwag	PIQA	WinoG	Avg.
Base	0.2884	0.6444	**0.4057**	0.7018	0.5635	0.5208
AdamW	0.2952	**0.6608**	0.4006	0.7013	**0.5691**	0.5254
Muon	0.2850	0.6604	0.4013	0.7018	0.5667	0.5230
Hybrid	**0.2961**	0.6604	0.4023	**0.7051**	0.5659	**0.5260**

2.5 Efficiency Metrics

Hardware consistency. All models were trained using the same hardware configuration. Nevertheless, experiments were performed on different servers, which may introduce slight variations in system load and runtime conditions. Consequently, the reported efficiency metrics

(e.g., training time, step time, peak memory usage, and tokens per second) may contain minor noise and should be interpreted with caution.

Table 2: Training efficiency comparison.

Method	Train time	Step time	Peak memory	Tokens/s
AdamW	1h 20m	3.12 s.	18.05 GB	20990
Muon	1h 18m	3.11 s.	17.38 GB	21019
Hybrid	1h 16m	3.03 s.	17.72 GB	21609

2.6 Convergence Analysis

Loss trajectories. To complement the downstream evaluation, we analyze the training dynamics of the compared optimizers over the full 1500-step budget. Because wall-clock measurements can be mildly affected by server-side noise, we focus here on convergence as a function of optimization step rather than elapsed time. Figure 1 plots the training loss for AdamW, Muon, and the Hybrid method. To improve readability and reduce high-frequency mini-batch noise, the raw per-step loss is smoothed with a moving-average window of 50 steps.

As observed in Figure 1, the loss trajectories for AdamW, Muon, and the Hybrid optimizer are remarkably similar, showing almost identical convergence rates over the 1500-step budget. We attribute this parity to the specific training regime used in our experiments. First, the relatively small model scale (Qwen 2.5 0.5B) presents a well-conditioned optimization landscape where the standard AdamW baseline is already highly efficient. Second, and more importantly, our effective batch size of 64 introduces a high degree of stochasticity into the gradient estimates. Advanced optimizers like Muon often require larger batch sizes (lower gradient noise) to effectively leverage matrix orthogonalization and structural momentum. In this high-noise regime, the benefits of the Muon update rule are largely masked, leading to performance that perfectly mirrors the highly robust AdamW baseline. Notably, however, the Hybrid and Muon methods match the baseline without introducing instability, confirming their safe applicability even in resource-constrained (small batch) environments.

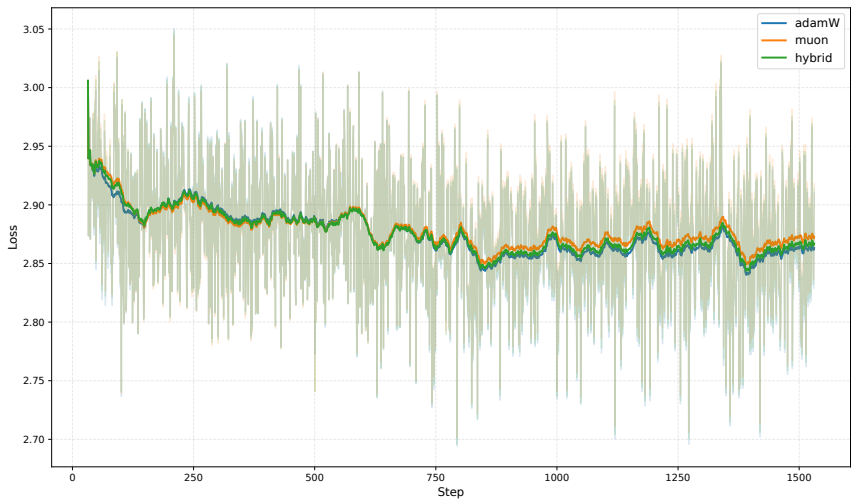

Figure 1: Training loss versus optimization step for the three optimizer configurations. Each curve is smoothed with a moving-average window of 50 steps

3 Discussion

Our results reveal that all three optimization strategies converge to comparable performance on both training loss and downstream benchmarks, with differences in average accuracy smaller than 0.3 percentage points. The Hybrid approach achieves the highest average score (0.5260), marginally outperforming AdamW (0.5254) and Muon (0.5230), yet these gaps fall well within the range of statistical noise given that each configuration was trained only once. We attribute the overall parity to two compounding factors. The Qwen2.5-0.5B model is sufficiently small that AdamW already navigates its optimization landscape efficiently, leaving little room for more sophisticated update rules to demonstrate an advantage. Furthermore, the effective batch size of 64 produces noisy gradient estimates that obscure the signal upon which Muon's orthogonal update relies; prior work has shown that Muon's benefits become more pronounced at larger batch sizes. Despite this, neither Muon nor the Hybrid method introduced training instability, suggesting that both are safe drop-in replacements for AdamW under the resource constraints.

4 Limitations

Several limitations constrain the conclusions that can be drawn from this study.

Single-server variance. Although all experiments targeted identical hardware, they were executed on physically distinct servers. Background system load, thermal throttling, and driver-level non-determinism can all influence wall-clock timings. This is most evident in the efficiency table, where the Hybrid optimizer reports a shorter training time and higher token throughput than both simpler methods despite managing three optimizer states - a result that is counter-intuitive and likely reflects favorable scheduling conditions rather than a genuine algorithmic advantage. Efficiency conclusions should therefore be treated as indicative rather than definitive, and ideally all runs should be repeated on a single, thermally stable machine with controlled system load.

Single training run per configuration. Each optimizer was trained exactly once with a fixed random seed. Without multiple independent runs, it is impossible to estimate the variance of the reported metrics or to determine whether observed differences in downstream accuracy are statistically meaningful. Best practice would involve at least three to five runs per configuration with different seeds, from which means and standard deviations can be reported. The current setup cannot distinguish genuine optimizer effects from seed-specific artifacts.

No hyperparameter search. The hyperparameters for each optimizer - most critically the learning rates - were set based on published recommendations rather than tuned for this specific model, dataset, and training budget. Muon in particular is sensitive to its learning rate and Newton–Schulz iteration count, and a systematic sweep might reveal configurations where it outperforms AdamW more clearly. As reported, the comparison is fair in the sense that all optimizers use broadly reasonable settings, but it may inadvertently favor AdamW, for which the chosen learning rate is well-established.

Limited training scale. At 1500 steps with an effective batch size of 64, the total number of training tokens is modest (<100M tokens). Optimizer differences that manifest over longer training horizons or at larger batch sizes may not be observable within this budget. Scaling the experiment to larger models, longer schedules, or higher batch sizes would be necessary to draw broader conclusions about the relative merits of Muon-based strategies.

Benchmark sensitivity. Zero-shot evaluation on five benchmarks, each of which has its own variance, compounds the difficulty of drawing firm conclusions from single-run results. Task-level scores such as ARC-Challenge can fluctuate by several tenths of a percent across evaluation runs depending on prompt formatting and tokenization details.

5 Conclusion

We presented a systematic comparison of three optimization strategies for continued pre-training of the Qwen2.5-0.5B language model: a standard AdamW baseline, a Moonlight Muon configuration that applies orthogonal gradient updates to internal 2D weight matrices, and a depth-stratified Hybrid approach that restricts Muon to the first 12 transformer layers while using AdamW for deeper layers and scalar parameters. Across five zero-shot reasoning benchmarks and 1500 training steps, all three methods converge at nearly identical rates and achieve comparable downstream accuracy, with the Hybrid approach reaching the highest aggregate score of 0.5260.

These findings suggest that Muon and Hybrid strategies are viable alternatives to AdamW that introduce no instability, even under resource-constrained conditions with small effective batch sizes. At the same time, they do not demonstrate a clear practical advantage over the well-established AdamW baseline in this regime, consistent with the hypothesis that Muon's orthogonalization benefits are most pronounced at larger batch sizes and model scales.

References

[1] Qwen Team. Qwen2.5-0.5B. https://huggingface.co/Qwen/Qwen2.5-0.5B

[2] Elriggs. openwebtext-100k. https://huggingface.co/datasets/Elriggs/openwebtext-100k

[3] EleutherAI. Language Model Evaluation Harness. https://github.com/EleutherAI/lm-evaluation-harness

[4] Moonshot AI. Moonlight / Muon optimizer. https://github.com/MoonshotAI/Moonlight